

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Высшая школа кибертехнологий, математики и статистики (факультет)
Направление: Информационные системы и технологии
Программа: Системы и технологии искусственного интеллекта (подготовка специалистов
топ-уровня)

О Т Ч Е Т
по учебной практике

Выполнил студент 1 курса
группы 15.27Д-ИСТ01/256
ВШКМиС (факультет)

(ФИО)

(подпись)

Проверил:

Доцент/профессор
Образовательного центра в сфере искусственного интеллекта

(оценка)

(подпись)

(дата)

Москва
2026

Оглавление

<u>ВВЕДЕНИЕ.....</u>	<u>3</u>
<u>1. ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ</u>	<u>5</u>
1.1. ОБЗОР АНАЛИТИЧЕСКОЙ ПЛАТФОРМЫ LOGINOM И ДАННЫХ INSTACART	5
1.2. РАЗРАБОТКА АНАЛИТИЧЕСКИХ ВЕБ-СЕРВИСОВ	6
1.3. ИНТЕГРАЦИЯ БОЛЬШОЙ ЯЗЫКОВОЙ МОДЕЛИ	9
1.4. ИНТЕГРАЦИЯ С FLASK-ПРИЛОЖЕНИЕМ И ИНТЕРФЕЙС.....	11
1.5. КЭШИРОВАНИЕ И ОПТИМИЗАЦИЯ ЗАПРОСОВ.....	15
1.6. ТЕСТИРОВАНИЕ И ПУБЛИКАЦИЯ В ОБЛАКЕ	16
<u>2. АНАЛИТИЧЕСКИЙ РАЗДЕЛ</u>	<u>18</u>
2.1. ОЦЕНКА ЭФФЕКТИВНОСТИ РАЗРАБОТАННОГО РЕШЕНИЯ.....	19
2.2. ПРЕИМУЩЕСТВА И ОГРАНИЧЕНИЯ LOW-CODE/NO-CODE И LLM-ПОДХОДА.....	20
2.3. СРАВНЕНИЕ ЗАПЛАНИРОВАННЫХ И ФАКТИЧЕСКИХ РЕЗУЛЬТАТОВ.....	20
2.4. РЕКОМЕНДАЦИИ ПО УЛУЧШЕНИЮ	21
<u>ЗАКЛЮЧЕНИЕ.....</u>	<u>22</u>
<u>ВКЛАД УЧАСТНИКОВ КОМАНДЫ.....</u>	<u>23</u>
<u>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....</u>	<u>25</u>

Введение

Цель практики — освоить технологию разработки клиентских аналитических сервисов на основе связки low-code/no-code аналитики, веб-приложения и большой языковой модели (LLM). В качестве предметной области выбран анализ покупательского поведения клиентов онлайн-магазина продуктов на данных датасета Instacart.

Для достижения цели в ходе практики решались следующие **задачи**:

1. Изучить аналитическую платформу Loginom и структуру датасета Instacart.
2. Разработать базовый аналитический сервис, который по идентификатору клиента возвращает историю его покупок и ключевые характеристики покупательского поведения.
3. Реализовать дополнительные клиентские аналитические сервисы, опубликованные как REST-сервисы.
4. Собрать веб-приложение с многостраничным пользовательским интерфейсом для запуска сервисов и просмотра результатов.
5. Интегрировать большую языковую модель для генерации понятных клиенту рекомендаций и пояснений.
6. Реализовать кэширование ответов языковой модели для оптимизации повторных запросов.
7. Улучшить интерфейс: навигация, визуальные элементы, диаграммы, изображение, сгенерированное нейросетью.
8. Провести тестирование сценариев работы и опубликовать приложение в облаке.

Используемые инструменты и технологические классы: среда визуальной аналитики Loginom; язык Python и микрофреймворк Flask; архитектурный стиль REST и формат JSON; большая языковая модель Mistral, подключаемая через библиотеку LangChain; реляционная база данных SQLite

(датасет Instacart); библиотека визуализации Chart.js; система контроля версий Git; облачная платформа Amvera; локальное файловое хранилище для кэша.

Актуальность темы. Подход low-code/no-code позволяет резко сократить время от идеи до работающего прототипа: визуальная сборка аналитических сценариев заменяет написание большого объёма кода. Одновременно большие языковые модели дают возможность превращать сухие числовые результаты в понятные человеку тексты и рекомендации. Комбинация этих технологий — современный способ быстро создавать минимально жизнеспособные продукты (MVP), что востребовано в текущей IT-индустрии.

Структура отчёта. Работа состоит из двух разделов. *Исследовательский раздел* описывает процесс работы: обзор платформы и данных, разработку аналитических сервисов, интеграцию языковой модели и Flask-приложения, механизм кэширования, тестирование и публикацию в облаке. *Аналитический раздел* содержит постановку задач методом How Might We, оценку эффективности, разбор преимуществ и ограничений подхода, сравнение плановых и фактических результатов и рекомендации. Завершают отчёт выводы, вклад участников команды и список источников.

1. Исследовательский раздел

1.1. Обзор аналитической платформы Loginom и данных Instacart

Loginom — отечественная low-code платформа для визуального проектирования сценариев обработки и анализа данных. Работа строится на *графе обработки*: аналитик размещает узлы (импорт данных, SQL-запрос, фильтрация, группировка, соединение таблиц и т. д.) и соединяет их потоками данных. Готовый сценарий можно *опубликовать как REST-сервис* — тогда к нему можно обращаться из внешних программ по HTTP, передавая входные параметры и получая результат в формате JSON.

Ключевые возможности платформы, использованные в работе:

- **Подключение к данным.** База Instacart в формате SQLite подключается как источник; к таблицам обращаются SQL-запросами внутри сценария.
- **Визуальная сборка сценариев.** Логика сервиса собирается из узлов без написания серверного кода.
- **Публикация веб-сервисов.** Каждый сценарий превращается в конечную точку REST с входной переменной `user_id` и табличным результатом (`DataSet.Rows`).

Сравнение с другими low-code/no-code средствами. Среди аналогов — KNIME и Alteryx (визуальный анализ данных), Power Query / Power BI Dataflows (подготовка данных в экосистеме Microsoft), Node-RED (потокковая интеграция). Loginom выгодно отличается ориентацией на бизнес-аналитику и **встроенной публикацией сценариев как REST-сервисов «из коробки»**, что критично для нашей задачи. По сравнению с «чистым» программированием (аналитика на pandas) Loginom проигрывает в гибкости сложной логики, но выигрывает в скорости сборки типовых сценариев.

Датасет Instacart описывает историю онлайн-заказов продуктов (более 3 млн заказов от более чем 200 000 клиентов). В работе используются следующие таблицы:

Таблица	Ключевые поля	Назначение
orders	order_id, user_id, order_number, order_dow, order_hour_of_day, days_since_prior_order	Заказы: номер по счёту, день недели, час, интервал с прошлым заказом
orders_prior	order_id, product_id, add_to_cart_order, reordered	Состав прошлых заказов; признак повторной покупки
products	product_id, product_name, aisle, department, department_rus	Справочник товаров с русским названием отдела

Между сущностями действуют связи «один-ко-многим»: один пользователь — много заказов, один заказ — много товаров, один товар — во многих заказах. На этих данных строятся все аналитические сервисы.

1.2. Разработка аналитических веб-сервисов

В проекте задействовано **пять** аналитических сервисов, объединённых в один пакет Loginom (instacart_ws_Abakarov1). Из них четыре опубликованы как REST-сервисы Loginom, а сервис «Отделы и категории» рассчитывается на стороне приложения из результатов базового сервиса. Все сервисы работают по единой схеме: вход — user_id, выход — табличный результат.

Этапы создания базового сервиса (GetUserHistory): подключение к базе Instacart → SQL-запрос с соединением таблиц и фильтрацией по user_id, подсчётом числа покупок каждого товара → сортировка и ограничение вывода top-10 → настройка входного параметра user_id → свёртка в подмодель и публикация как REST-сервис.

Реализованные сервисы и их выходные поля:

Сервис	Назначение	Выходные поля	Источник
GetUserHistory	Топ-10 частых товаров	product_name, aisle, department, department_rus, order count	Loginom
GetForgottenProducts	Привычные товары, которых не было в последних заказах	product_name, department_rus, aisle, times_bought, orders_since_last	Loginom
GetPurchaseRhythm	Ритм заказов (частота, интервал, день, час)	total_orders, avg_days_between, favorite_dow, favorite_hour	Loginom
GetClientSummary	Сводные метрики поведения	total_orders, unique_products, total_items, reorder_rate, top_department_rus	Loginom
Отделы и категории	Распределение покупок по отделам с долями	department_rus, order_count, share	Расчёт в приложении

Из этих сервисов два являются **самостоятельными идеями команды** (сводка по клиенту и распределение по отделам), а остальные развивают предложенные постановки задачи. Сервис «Забытые товары» реализует полезную бизнес-механику: сравнивает историческую частоту покупок товара с номером последнего заказа клиента и находит товары, которые покупались часто, но отсутствовали в свежих заказах, — это позволяет мягко напомнить клиенту о том, что он мог забыть.

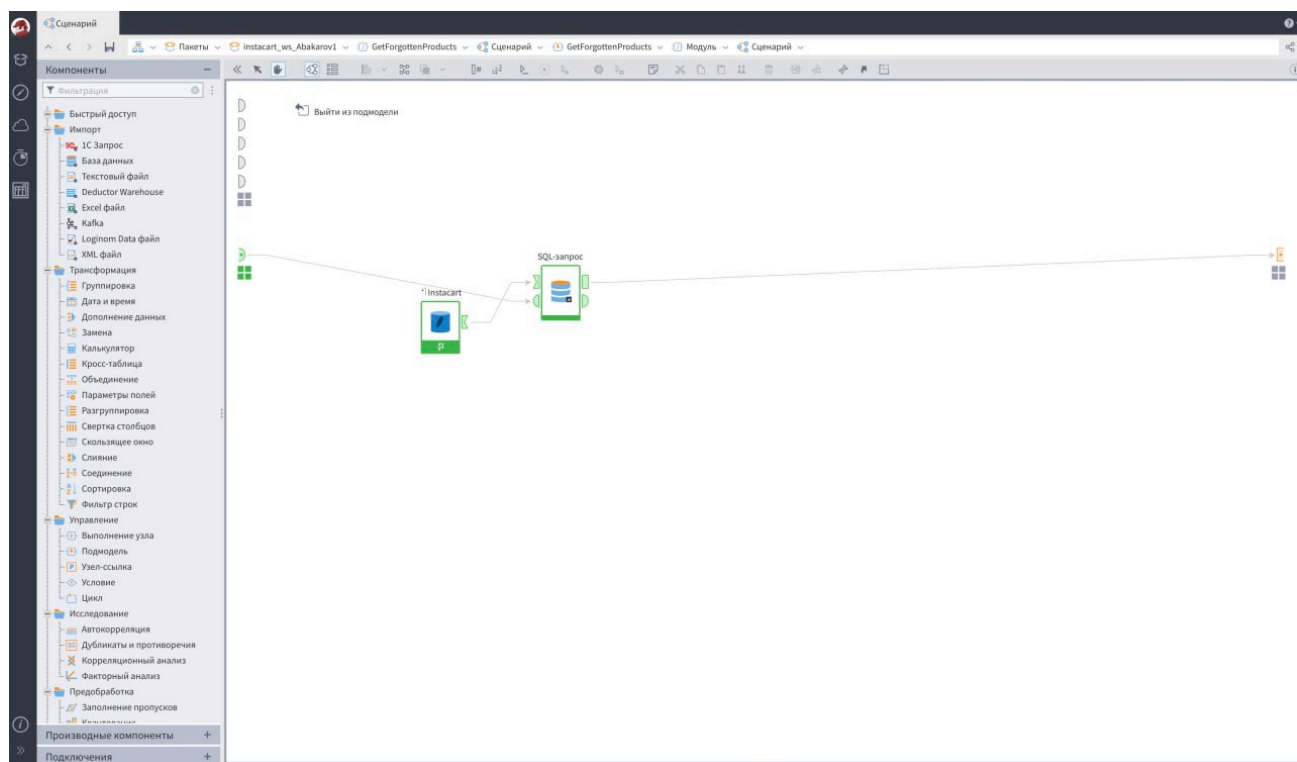


Рисунок 1. Сценарий сервиса GetForgottenProducts в Loginom.)

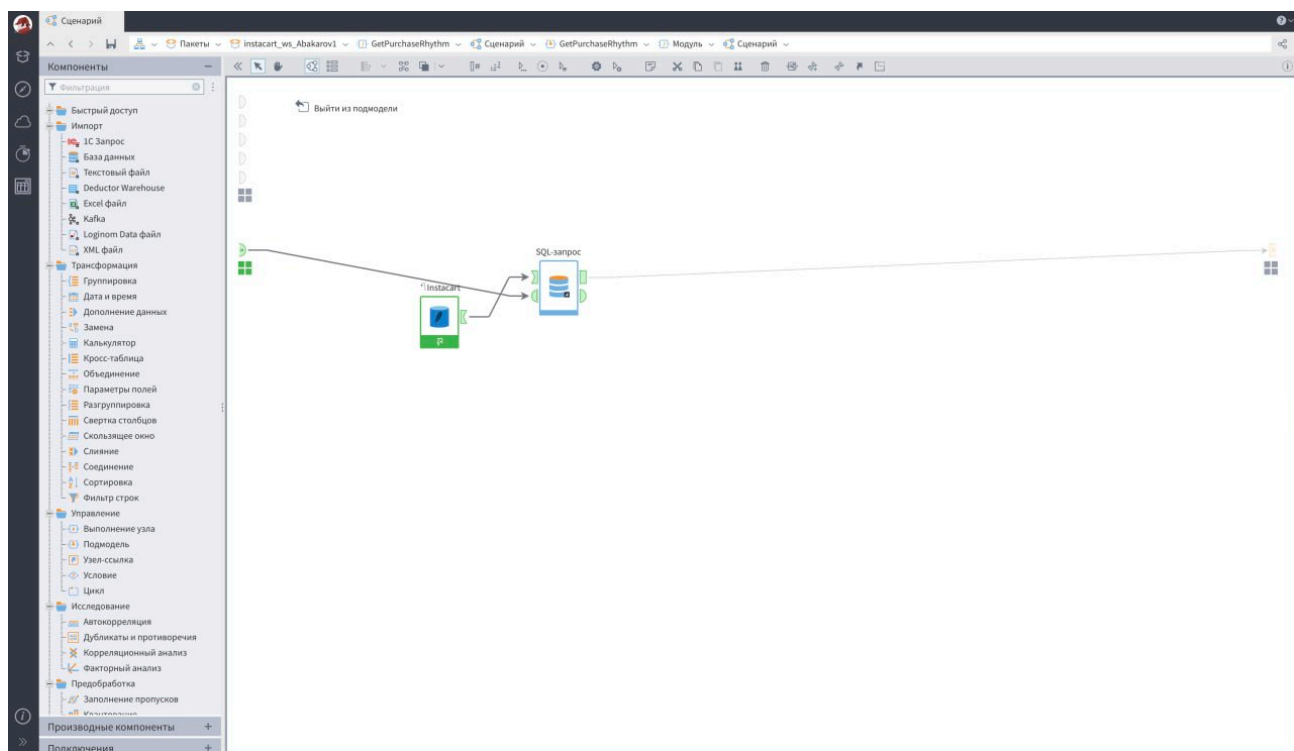


Рисунок 2. Сценарий сервиса GetPurchaseRhythm в Loginom

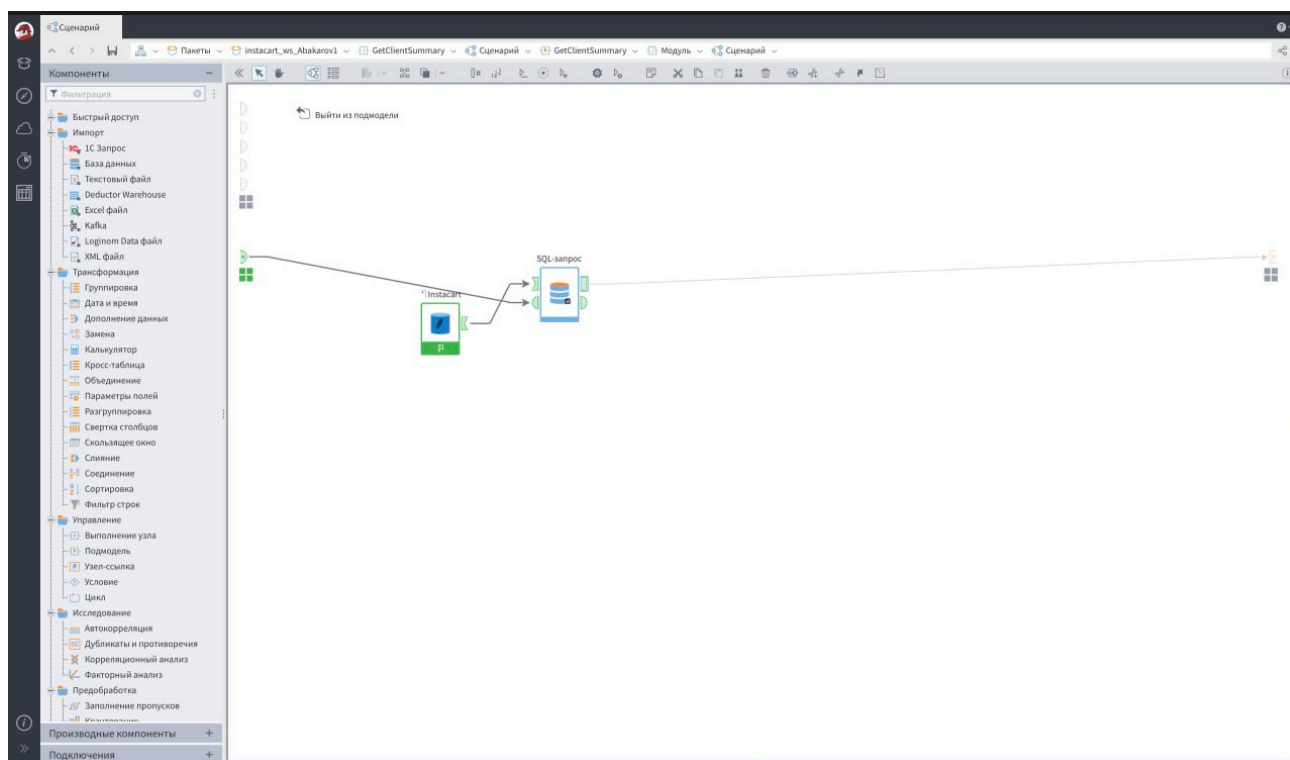
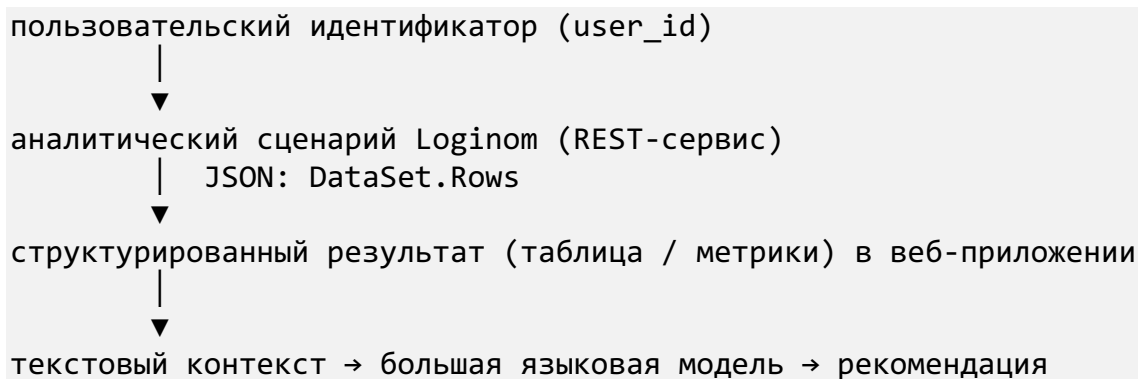


Рисунок 3. Сценарий сервиса GetClientSummary в Loginom

Схема взаимодействия одина для всех сервисов:



1.3. Интеграция большой языковой модели

Для генерации текстовых рекомендаций используется языковая модель **Mistral** (mistral-large-latest), подключаемая через библиотеку **LangChain** (пакет langchain-mistralai). Изначально рассматривалась модель GigaChat, однако по соображениям доступности и стабильности API команда перешла на

Mistral (замена LLM допускается по согласованию с преподавателем). Ключ доступа хранится в файле окружения `.env` и не попадает в репозиторий.

Подготовка данных для LLM. Табличный результат Loginom преобразуется в компактный текстовый контекст. Например, для истории покупок формируется нумерованный список товаров:

```
def build_products_text(rows):
    lines = []
    for i, row in enumerate(rows, start=1):
        lines.append(
            f"{i}. {row['product_name']} – "
            f"отдел: {row['department_rus']}, "
            f"категория: {row['aisle']}, "
            f"заказов: {row['order_count']}"
        )
    return "\n".join(lines)
```

Параметры и промпты. Модель настроена на дружелюбный, краткий стиль (`temperature = 0.3`, `max_tokens = 500`). Каждому сервису задан свой системный и пользовательский промпт. Пример системного промпта базового сервиса:

«Ты — персональный помощник покупателя в продуктовом онлайн-магазине. Твоя задача — анализировать историю покупок клиента и давать ему понятные, дружелюбные рекомендации прямо в его личном кабинете. Обращайся к клиенту на "вы", тепло и по-человечески. Пиши кратко — не более 5–6 предложений.»

Примеры рекомендаций (клиент `user_id = 1`):

- *История покупок:* «Здравствуйте! Заметили, что вы часто выбираете полезные перекусы: орешки, вяленое мясо и сыр...»
- *Ритм покупок:* «Здравствуйте! Вы обычно делаете покупки примерно раз в три недели, чаще всего по четвергам утром...»
- *Отделы:* «У вас ярко выражена склонность к быстрым перекусам и напиткам — почти две трети покупок приходятся на снеки и напитки...»

Роль LLM в проекте. Модель интерпретирует числовые результаты, генерирует персональные рекомендации и поясняет выводы на естественном языке, превращая «сухую» аналитику Loginom в понятный клиенту текст.

1.4. Интеграция с Flask-приложением и интерфейс

Веб-приложение построено на микрофреймворке **Flask** и состоит из маршрутов (`app.py`), бизнес-логики (`business-rules.py`) и HTML-шаблона (`templates/index.html`).

Структура и маршруты. Все сервисы описаны в едином реестре `SERVICES`, где указаны заголовок, имена функций бизнес-логики, способ отображения (таблица или метрики), колонки и параметры диаграммы. Один обобщённый обработчик обслуживает любой сервис, что упрощает добавление новых. Приложение **многостраничное**:

Маршрут	Метод	Назначение
/	GET	Главная страница — личный кабинет, баннер, навигация
/history	GET/POST	Раздел «Мои покупки» — топ-10 товаров и рекомендация
/insights	GET	Раздел «Аналитика» — забытые товары, ритм, сводка, отделы
/service/<key>	POST	Запуск сервиса, HTML-результат
/api/service/<key>	POST	Тот же сервис в виде JSON (REST)
/login, /logout	POST	Вход по <code>user_id</code> и выход

REST-взаимодействие с Loginom реализовано единой функцией:

```
LOGINOM_BASE =
"https://edu.loginom.dev/lgi/rest/instacart_ws_Abakarov1/"

def _call_loginom(service: str, user_id: int):
```

```
payload = {"Variables": {"user_id": user_id}}
response = requests.post(LOGINOM_BASE + service, json=payload,
timeout=30)
response.raise_for_status()
return response.json()["DataSet"]["Rows"]
```

Пользовательский интерфейс оформлен как «личный кабинет клиента» с верхней навигацией по трём разделам. Результат сервиса выводится либо таблицей со столбчатой диаграммой, либо плитками-метриками; ниже — текст промпта и ответ модели с отметкой «из кэша / новый запрос» и временем ответа. Предусмотрены понятные сообщения об ошибках. Визуальный стиль — зелёно-белая палитра «в духе продуктового магазина», карточки со скруглениями и тенями, шрифт Nunito (Google Fonts), текстовый логотип и маскот.

Диаграммы. Для наглядных графиков подключена библиотека **Chart.js** (через CDN): данные передаются из Flask в шаблон и вставляются в JavaScript-блок. Для сервиса «Отделы и категории» строится кольцевая диаграмма долей отделов, для топа товаров — столбчатая.

Визуализация с помощью нейросети. В шапку главной страницы добавлено изображение, сгенерированное нейросетью (корзина со свежими продуктами в фирменной зелёно-белой гамме). Использованный промпт: «Уютная корзина со свежими продуктами (овощи, фрукты, молоко, хлеб) на светлом фоне, минимализм, мягкий свет, зелёно-белая палитра, широкий баннер для сайта доставки продуктов, без текста». Изображение подаётся из папки **static** и подхватывается приложением автоматически.

Личный кабинет клиента Instacart

Аналитика Loginom + Flask + рекомендации Mistral



Шаг 1

Авторизация клиента

Введите user_id

Авторизоваться

Клиент не выбран

Разделы

Куда дальше

Мои покупки →

Топ-10 товаров и персональная рекомендация.

Аналитика →

Забутые товары, ритм покупок, сводка и отдели.

Рисунок 4. Главная страница приложения — личный кабинет с баннером

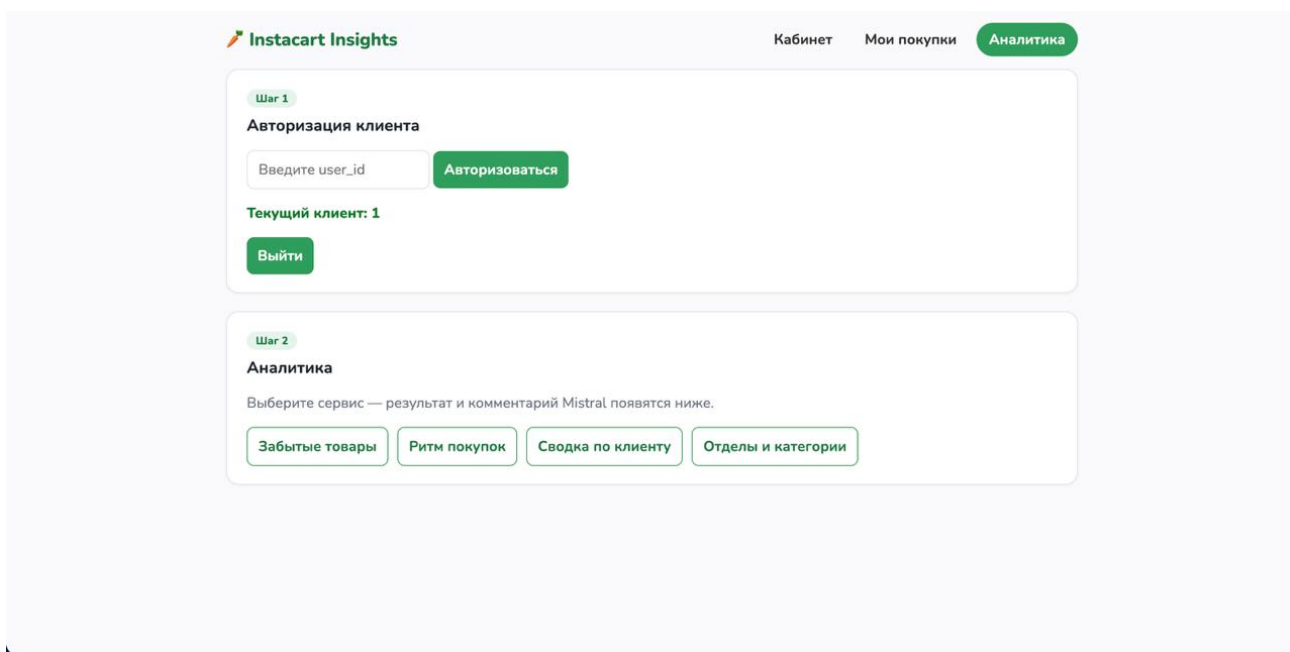


Рисунок 5. Раздел «Аналитика» с кнопками сервисов

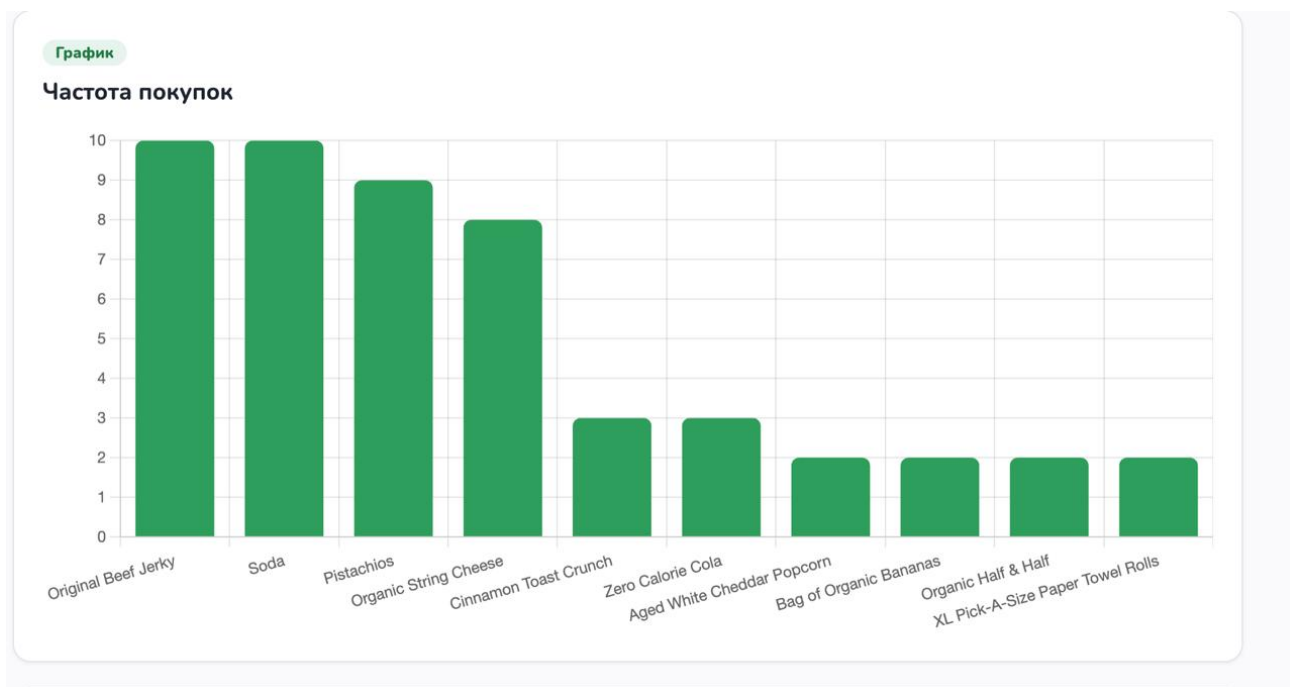


Рисунок 6. Результат сервиса: таблица и диаграмма Chart.js

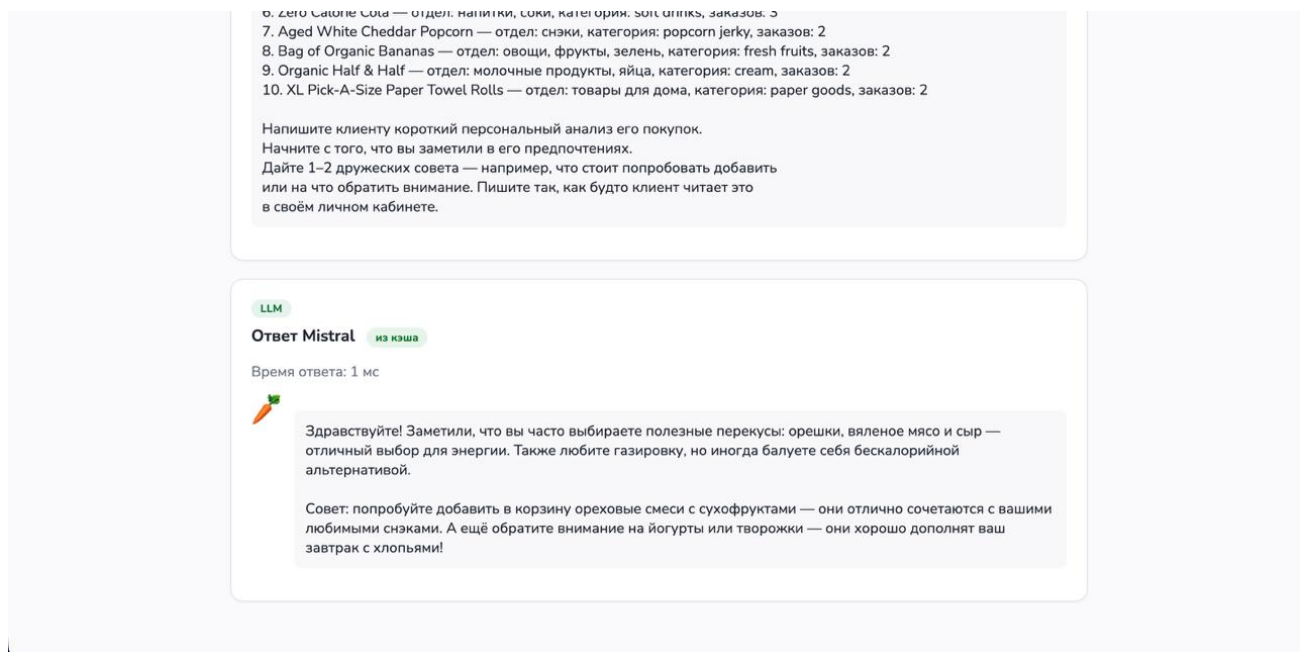


Рисунок 7. Ответ Mistral с индикатором кэша — вставьте скриншот.

1.5. Кэширование и оптимизация запросов

Обращение к языковой модели — самая медленная операция в цепочке (несколько секунд). Чтобы не запрашивать модель повторно при одинаковых входных данных, реализовано **кэширование ответов**.

Логика кэширования. Для каждого запроса вычисляется ключ — SHA-256 хеш от имени сервиса, идентификатора клиента и текстового контекста. Перед обращением к модели проверяется наличие ответа по ключу; если он есть — возвращается сразу.

```
def _cached_llm(cache_key, system_prompt, user_prompt):
    cache = _load_cache()
    cached = cache.get(cache_key)
    if cached is not None:
        # ответ уже есть — берём из кэша
        return cached["prompt_text"], cached["recommendation"], True
    result = llm.invoke([SystemMessage(content=system_prompt),
                        HumanMessage(content=user_prompt)])
    cache[cache_key] = {"prompt_text": user_prompt, "recommendation":
result.content}
    _save_cache(cache)
    # сохраняем новый ответ
    return user_prompt, result.content, False
```

Схема «клиент — приложение — LLM»:

```
клиент → приложение → [есть в кэше?] → да → быстрый ответ из файла
                                     → нет → запрос к Mistral →
сохранить в кэш → ответ
```

Структура хранилища. Кэш хранится в файле `llm_cache.json` (словарь «ключ → {промт, ответ}»). Файл лежит на диске, а не в памяти процесса, поскольку модуль бизнес-логики перезагружается на каждый запрос; дисковое хранилище гарантирует, что кэш переживает перезагрузку. Файл исключён из системы контроля версий. Эффект подтверждён замерами: первый запрос ~4,3 с, повторный — около 10 мс (ускорение более чем в 300 раз).

1.6. Тестирование и публикация в облаке

Тестирование проводилось на двух уровнях: автоматические тесты и ручная проверка сквозных сценариев (локально и в облаке).

Автоматические тесты написаны с использованием `pytest`; обращения к `Loginom` и к языковой модели заменяются заглушками (`mock`), поэтому тесты не требуют сети и ключей. Всего **25 тестов**, все проходят успешно. Набор покрывает: загрузку страниц и навигацию, авторизацию и валидацию `user_id`, обработку неизвестного клиента, запуск каждого из пяти сервисов и корректный рендеринг таблиц, метрик и диаграмм, **работу кэша, разделение клиентов** (данные не смешиваются), обработку сбоя сервиса и пустого ответа.

Ручная проверка сквозных сценариев выполнена на «живых» сервисах `Loginom` и реальной модели `Mistral`. Замеры времени отклика (клиент `user_id = 1`):

Сервис	Время (первый запрос)	Время (из кэша)
История покупок	4268 мс	~12 мс
Забытые товары	2883 мс	~1 мс
Ритм покупок	2571 мс	~1 мс
Сводка по клиенту	3867 мс	~1 мс
Отделы и категории	3836 мс	~1 мс

Пример корректности данных для клиента `user_id = 1`: 11 заказов, 18 уникальных товаров, доля повторных покупок 69 %, любимый отдел — «снэки», средний интервал ~19 дней, чаще всего заказы по четвергам около 8 часов. Текстовые выводы модели согласуются с этими цифрами, что подтверждает корректность связки «аналитика → LLM».

Публикация в облаке. Приложение развёрнуто на платформе Amvera и доступно по адресу <https://practica2-gekyme.amvera.io/>. Для облака порт читается из переменной окружения `PORT`, приложение слушает `0.0.0.0`, а секреты передаются через переменные окружения Amvera. Проверка показала, что сценарии работают в облаке так же, как локально.

Обнаруженные проблемы и решения. Имя файла бизнес-логики содержит дефис, поэтому модуль загружается через `importlib`; кэш хранится на диске из-за перезагрузки модуля; для отображения дня недели согласована числовая кодировка `order_dow` с текстовыми названиями. Все проблемы устранены.

2. Аналитический раздел

2.1. Постановка задач методом How Might We (HMW)

Для обоснования состава сервисов применён метод дизайн-мышления **How Might We** («Как мы могли бы...»). Его суть — переформулировать выявленную проблему пользователя в открытый вопрос-возможность, начинающийся со слов «Как мы могли бы...». Такой вопрос достаточно широк, чтобы допускать разные решения, и одновременно достаточно конкретен, чтобы направлять разработку: каждый вопрос HMW становится основанием для отдельного сервиса, а ответом на него служит реализованная функция.

Продуктовый вопрос верхнего уровня формулируется так:

Как мы могли бы превратить обезличенные числовые данные о покупках в понятную и полезную для клиента информацию прямо в личном кабинете?

Ответ на него — вся связка «аналитика Loginom → метрики → дружелюбный текст Mistral». Далее вопрос декомпозируется на частные HMW, каждый из которых закрывается отдельным сервисом:

Проблема клиента	Вопрос How Might We	Сервис-ответ
Не помнит, что покупает чаще всего, и не получает советов	Как мы могли бы показать клиенту его ключевые предпочтения и дать персональный совет?	Топ-10 товаров
Забывает вовремя докупить привычные товары	Как мы могли бы вовремя напомнить о привычных товарах, которых давно не было в заказах?	Забутые товары
Не планирует момент следующего заказа	Как мы могли бы помочь клиенту понять, когда ему снова понадобится оформить заказ?	Ритм покупок

Не видит общую картину своего поведения	Как мы могли бы дать клиенту наглядный «портрет» его покупок одним экраном?	Сводка по клиенту
Не осознаёт структуру своей корзины	Как мы могли бы показать клиенту, покупателем какого типа он является по составу покупок?	Отделы и категории

Такой подход обеспечивает **прослеживаемость решений**: каждый сервис отвечает на конкретный сформулированный вопрос-возможность, а не появляется произвольно. NMW не требует полевого опроса и потому уместен на этапе учебного прототипа; при дальнейшем развитии продукта его логично дополнить проверкой сформулированных гипотез на реальных пользователях.

2.2. Оценка эффективности разработанного решения

Качество результатов. Все пять сервисов возвращают корректные и полезные для клиента данные. Имена и типы полей, возвращаемых Loginom, полностью совпали с ожиданиями приложения, что говорит о согласованности контракта между аналитикой и приложением.

Скорость отклика. Основной вклад во время ответа вносит обращение к языковой модели (2,5–4,3 с); обращение к Loginom и рендеринг занимают доли секунды. Кэширование снижает время повторного ответа до ~1–12 мс, делая интерфейс отзывчивым.

Удобство интерфейса. Сценарий целостен: клиент вводит идентификатор один раз, переходит между разделами кабинета и запускает любой сервис одной кнопкой, сразу видя и числовой результат (в т. ч. диаграмму), и текстовую интерпретацию. Отметка кэша и время ответа делают работу системы прозрачной.

2.3. Преимущества и ограничения low-code/no-code и LLM-подхода

Преимущества: высокая скорость прототипирования (сервисы собираются визуально, добавление нового в приложение — одна запись в реестре SERVICES); наглядность графа обработки; быстрая интеграция аналитики и текста через LLM; доступность для начинающих разработчиков.

Ограничения: сложную нестандартную логику труднее реализовать в визуальной среде; работа зависит от доступности сервера Loginom и API Mistral; требуется строгое согласование контракта (имена сервисов и колонок); ответы LLM недетерминированы (частично компенсируется низкой «температурой» и кэшированием).

Примеры задач, трудных для одного инструмента. Только визуальной аналитикой невозможно сгенерировать связный дружелюбный текст рекомендации; только языковой моделью — надёжно и воспроизводимо посчитать метрики по большой базе. Задачу решает именно связка «аналитика + LLM».

2.4. Сравнение запланированных и фактических результатов

Реализовано полностью: базовый и четыре дополнительных сервиса (из них два — самостоятельные идеи команды); многостраничное веб-приложение с навигацией; интеграция Mistral с индивидуальными промптами; кэширование ответов; диаграммы Chart.js; изображение, сгенерированное нейросетью; автоматическое (25 тестов) и ручное тестирование; публикация в облаке Amvera.

Скорректировано: вместо GigaChat использована Mistral (доступность и стабильность API); архитектура обобщена (единый обработчик и реестр сервисов) вместо отдельного кода на каждый сервис.

Сложности: согласование контракта Loginom ↔ приложение, загрузка модуля с дефисом в имени, выбор дискового кэша, кодировка дня недели. Все преодолены (см. 1.6).

2.5. Рекомендации по улучшению

- Вынести расчёт «Отделы и категории» в отдельный веб-сервис Loginom для единообразия архитектуры.
- Перевести кэш с файла на таблицу SQLite и ввести срок жизни записей.
- Добавить больше визуализаций (тепловая карта времени заказов, динамика покупок).
- Повысить надёжность: повторные попытки и «мягкая» деградация при недоступности внешних сервисов.
- Дополнить UX пояснениями «почему это рекомендуется» и итоговым блоком рекомендаций.

Заключение

В ходе учебной практики освоена технология создания клиентских аналитических сервисов на стыке low-code аналитики (Loginom), веб-разработки (Python/Flask, REST) и больших языковых моделей (Mistral через LangChain).

Основные итоги:

- Изучены платформа Loginom, архитектурный стиль REST и датасет Instacart.
- Разработаны пять аналитических сервисов и многостраничное веб-приложение, объединяющее их в единый личный кабинет клиента.
- Настроена интеграция с языковой моделью, превращающая числовые результаты в понятные рекомендации.
- Реализовано кэширование ответов модели, ускоряющее повторные запросы более чем в 300 раз.
- Добавлены диаграммы Chart.js и изображение, сгенерированное нейросетью.
- Проведено тестирование (25 автоматических тестов и ручная проверка), приложение развёрнуто в облаке Amvera.

Выводы. Связка low-code/no-code аналитики, веб-сервисов и языковых моделей позволяет за короткий срок собрать работающий MVP, объединяющий расчёты и их человекочитаемую интерпретацию. Подход особенно эффективен для быстрых прототипов, но требует аккуратной интеграции компонентов и имеет ограничения по гибкости логики.

Личный прогресс. Развиты навыки визуальной аналитики в Loginom, проектирования REST-взаимодействия, разработки веб-приложений на Flask, интеграции языковых моделей и работы с промптами, а также командной разработки с использованием Git. Задачи практики выполнены в полном объёме.

Вклад участников команды

Участник	Роль	Вклад в проект
Абакаров Магомед Шамилевич	Аналитик / Loginom	Спроектировал и опубликовал аналитические веб-сервисы Loginom (GetUserHistory, CheckUserId, GetForgottenProducts, GetPurchaseRhythm, GetClientSummary); написал SQL-запросы и настроил входные/выходные параметры и публикацию как REST; сформировал промпты к языковой модели по каждому сервису; подготовил презентацию командного проекта.
Черевиченко Николай Александрович	Бэкенд / Flask + интеграции	Разработал Flask-приложение и обобщённый обработчик сервисов; реализовал REST-взаимодействие с Loginom и интеграцию с Mistral; реализовал кэширование ответов модели; добавил маршруты и API-ответы для фронтенда; покрыл проект автоматическими тестами; подготовил приложение к деплою и публикации в Amvera.
Поляков Максим Константинович	Фронтенд / UX	Спроектировал интерфейс личного кабинета: страницы и навигацию, форму

		user_id, таблицы, плитки-метрики и блок рекомендаций; подключил диаграммы Chart.js; добавил индикатор кэша; интегрировал сгенерированное нейросетью изображение; оформил единый визуальный стиль (палитра, шрифты, карточки, бренд).
--	--	---

Список использованных источников

1. Loginom. Официальная документация. — URL: <https://loginom.ru/documentation> (дата обращения: 07.2026).
2. The Instacart Online Grocery Shopping Dataset 2017. — URL: <https://www.kaggle.com/c/instacart-market-basket-analysis> (дата обращения: 07.2026).
3. Flask Documentation. — URL: <https://flask.palletsprojects.com/> (дата обращения: 07.2026).
4. LangChain Documentation. — URL: <https://python.langchain.com/> (дата обращения: 07.2026).
5. Mistral AI. API Documentation. — URL: <https://docs.mistral.ai/> (дата обращения: 07.2026).
6. Chart.js Documentation. — URL: <https://www.chartjs.org/docs/> (дата обращения: 07.2026).
7. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures : дис. — University of California, Irvine, 2000.
8. Amvera Cloud. Документация по развёртыванию приложений. — URL: <https://docs.amvera.ru/> (дата обращения: 07.2026).